

Analytical Problem Solving

Warehouse Delivery Robot – 5×5 Grid Problem Solution

Problem Parameter	Specification
Grid size	5 × 5
Initial position	(1,1)
Delivery/goal position	(5,5)
Blocked cells	(2,2), (2,3), (3,3), (4,2), (4,4)
Allowed movements	Up, Down, Left, Right
Move cost	1 per move
Search algorithms	Breadth-First Search (BFS) and Depth-First Search (DFS)

Introduction

This problem models an autonomous delivery robot moving through a warehouse represented by a 5×5 grid. The robot begins at cell (1,1) and must deliver a package to cell (5,5). Five cells are occupied by shelving units and therefore cannot be entered: (2,2), (2,3), (3,3), (4,2), and (4,4). The robot can move only vertically or horizontally, and every legal move has a uniform cost of 1.

The problem is a classical state-space search problem. The robot must determine a sequence of actions that takes it from the initial state to the goal state without entering blocked cells. Because every movement has the same cost, BFS has an important advantage: when it finds the goal in an unweighted graph, the first solution is guaranteed to contain the minimum number of moves.

(a) Model this scenario as a formal intelligent agent problem. Clearly define the agent's PEAS description (Performance measure, Environment, Actuators, Sensors) for this warehouse robot.

PEAS stands for Performance measure, Environment, Actuators, and Sensors. It provides a formal description of what an intelligent agent must achieve, where it operates, how it can act, and what information it can perceive.

PEAS Component	Detailed Specification
Performance Measure	Successfully deliver the package to (5,5); avoid blocked/shelving cells; minimize the number of

PEAS Component	Detailed Specification
	movements; avoid illegal moves/collisions; complete the delivery reliably and efficiently.
Environment	A 5×5 discrete warehouse grid. The robot starts at (1,1), the goal is (5,5), and the blocked cells are (2,2), (2,3), (3,3), (4,2), and (4,4). The environment is static and deterministic for this problem, with uniform action cost 1.
Actuators	Robot movement mechanisms that execute Up, Down, Left, and Right actions. A delivery actuator or mechanism can be used to place/hand over the package at the goal cell.
Sensors	Position/location sensor, grid/map or obstacle sensor, and goal/package status sensor. In the abstract problem, the robot is assumed to know enough information to determine its current cell and whether a neighboring cell is blocked.

a.1 Performance Measure in More Detail

- Primary objective: reach the delivery location (5,5).
- Safety objective: never enter a cell occupied by shelving units.
- Efficiency objective: use as few movements as possible because every movement costs 1.
- Correctness objective: deliver the package at the specified goal cell.
- Operational objective: avoid unnecessary exploration when a valid route is available.

a.2 Environment Characteristics

Property	Classification	Reason
Observability	Fully observable (assumption)	The grid and blocked cells are specified to the agent.
Number of agents	Single-agent	Only the delivery robot makes decisions in the model.
Determinism	Deterministic	A legal movement always changes the robot's cell according to the defined transition.
Episodic/Sequential	Sequential	Each movement affects the robot's current state and future options.
Static/Dynamic	Static	Shelving locations do not change during the search.
Discrete/Continuous	Discrete	States are grid cells and actions are four discrete movement actions.
Known/Unknown	Known (assumption)	The map and movement rules are given before search.

(b) Classify the type of agent (simple reflex, model-based, goal-based, or utility-based) best suited for this task and justify your choice with reasoning specific to this scenario.

The best classification for this task is a Goal-Based Agent. A goal-based agent selects actions by considering whether they help the agent reach a specified goal. Here, the goal is explicit: move the robot from (1,1) to (5,5) while respecting the blocked cells.

Agent Type	Suitability	Reason
Simple Reflex Agent	Not best	It reacts to the current percept using condition-action rules. It does not naturally plan a multi-step route around several obstacles.
Model-Based Agent	Useful but not sufficient alone	It can maintain an internal representation of the environment, which is helpful, but the central requirement here is planning toward a goal.
Goal-Based Agent	Best suited	It explicitly searches for a sequence of actions that reaches (5,5), taking blocked cells and movement rules into account.
Utility-Based Agent	Possible but unnecessary	Utility is useful when several objectives must be balanced, such as speed, energy, risk, and cost. The stated problem mainly requires reaching the goal with unit cost.

A model-based goal-directed implementation would also be realistic for a physical warehouse robot, but among the four categories named in the question, Goal-Based Agent is the clearest answer because the task is explicitly defined by a destination goal and a search for an action sequence.

(c) Formally define this as a search problem by specifying: Initial State, Goal State, Actions, Transition Model, and Path Cost function.

The warehouse delivery task can be represented as a state-space search problem $P = (S, A, \text{Result}, s_0, G)$, together with a path-cost function.

Element	Formal Definition	Explanation
State Space S	All non-blocked cells in the 5×5 grid	A state is represented by the robot's current coordinate (r,c), where $1 \leq r, c \leq 5$ and the cell is not blocked.

Element	Formal Definition	Explanation
Initial State s_0	(1,1)	The robot starts at row 1, column 1.
Goal State G	{(5,5)}	The search succeeds when the robot reaches the package-delivery cell (5,5).
Actions A(s)	{Up, Down, Left, Right} subject to legality	An action is legal only when the destination remains inside the 5×5 grid and is not blocked.
Transition Model Result(s,a)	Move from s to the adjacent legal cell specified by a	For example, Result((1,1), Down)=(2,1). An action that enters a blocked/out-of-grid cell is not allowed.
Path Cost g(n)	$g(\text{child})=g(\text{parent})+1$	Every movement has cost 1, so the total path cost equals the number of moves.

c.1 Blocked Cells

Blocked/shelving cells are: (2,2), (2,3), (3,3), (4,2), and (4,4). These states are excluded from the reachable state space.

c.2 Grid Representation

Row \ Col	1	2	3	4	5
1	S	.	.	.	.
2	.	X	X	.	.
3	.	.	X	.	.
4	.	X	.	X	.
5	.	.	.	.	G

Legend: S = Start, G = Goal, X = Blocked/Shelf, . = Free cell.

(d) Apply Breadth-First Search (BFS) to find a path from (1,1) to (5,5). Show the step-by-step expansion of nodes (the frontier/queue at each step) until the goal is reached, and state the final path along with its total cost.

BFS explores the state space level by level. It uses a FIFO queue: the earliest discovered node is expanded first. Because every movement has the same cost (1), BFS is guaranteed to find a minimum-cost path in this problem.

For reproducibility, the neighbor generation order used below is: Down, Right, Up, Left. Already-discovered states are not inserted again. The queue shown in each row is the queue before the current node is expanded, followed by the nodes generated and the resulting queue.

Step	Expanded	Queue before expansion	Newly generated	Queue after expansion
1	(1,1)	[(1,1)]	[(2,1), (1,2)]	[(2,1), (1,2)]
2	(2,1)	[(2,1), (1,2)]	[(3,1)]	[(1,2), (3,1)]
3	(1,2)	[(1,2), (3,1)]	[(1,3)]	[(3,1), (1,3)]
4	(3,1)	[(3,1), (1,3)]	[(4,1), (3,2)]	[(1,3), (4,1), (3,2)]
5	(1,3)	[(1,3), (4,1), (3,2)]	[(1,4)]	[(4,1), (3,2), (1,4)]
6	(4,1)	[(4,1), (3,2), (1,4)]	[(5,1)]	[(3,2), (1,4), (5,1)]
7	(3,2)	[(3,2), (1,4), (5,1)]	[]	[(1,4), (5,1)]
8	(1,4)	[(1,4), (5,1)]	[(2,4), (1,5)]	[(5,1), (2,4), (1,5)]
9	(5,1)	[(5,1), (2,4), (1,5)]	[(5,2)]	[(2,4), (1,5), (5,2)]
10	(2,4)	[(2,4), (1,5), (5,2)]	[(3,4), (2,5)]	[(1,5), (5,2), (3,4), (2,5)]
11	(1,5)	[(1,5), (5,2), (3,4), (2,5)]	[]	[(5,2), (3,4), (2,5)]
12	(5,2)	[(5,2), (3,4), (2,5)]	[(5,3)]	[(3,4), (2,5), (5,3)]
13	(3,4)	[(3,4), (2,5), (5,3)]	[(3,5)]	[(2,5), (5,3), (3,5)]
14	(2,5)	[(2,5), (5,3), (3,5)]	[]	[(5,3), (3,5)]
15	(5,3)	[(5,3), (3,5)]	[(5,4), (4,3)]	[(3,5), (5,4), (4,3)]
16	(3,5)	[(3,5), (5,4), (4,3)]	[(4,5)]	[(5,4), (4,3), (4,5)]
17	(5,4)	[(5,4), (4,3), (4,5)]	[(5,5)]	[(4,3), (4,5), (5,5)]
18	(4,3)	[(4,3), (4,5), (5,5)]	[]	[(4,5), (5,5)]
19	(4,5)	[(4,5), (5,5)]	[]	[(5,5)]
20	(5,5)	[(5,5)]	[]	[]

d.1 BFS Final Path

BFS returns the following shortest path:

$(1,1) \rightarrow (2,1) \rightarrow (3,1) \rightarrow (4,1) \rightarrow (5,1) \rightarrow (5,2) \rightarrow (5,3) \rightarrow (5,4) \rightarrow (5,5)$

Number of moves / total path cost = 8.

This path is valid because every consecutive pair of cells differs by one horizontal or vertical move, and none of the visited cells is blocked.

(e) Apply Depth-First Search (DFS) to the same problem. Show the step-by-step expansion of nodes (the stack at each step) until the goal is reached, and state the final path along with its total cost.

DFS explores one branch as deeply as possible before backtracking. It uses a LIFO stack. DFS does not guarantee a shortest path in general. Its result depends strongly on the order in which neighboring states are considered.

For a deterministic trace, the same conceptual neighbor preference is used: Down, Right, Up, Left. Because a stack is LIFO, newly generated neighbors are pushed in reverse order so that Down is examined first, followed by Right, Up, and Left. States are marked discovered when pushed.

Step	Expanded	Stack before expansion	Newly generated	Stack after expansion
1	(1,1)	[(1,1)]	[(1,2), (2,1)]	[(1,2), (2,1)]
2	(2,1)	[(1,2), (2,1)]	[(3,1)]	[(1,2), (3,1)]
3	(3,1)	[(1,2), (3,1)]	[(3,2), (4,1)]	[(1,2), (3,2), (4,1)]
4	(4,1)	[(1,2), (3,2), (4,1)]	[(5,1)]	[(1,2), (3,2), (5,1)]
5	(5,1)	[(1,2), (3,2), (5,1)]	[(5,2)]	[(1,2), (3,2), (5,2)]
6	(5,2)	[(1,2), (3,2), (5,2)]	[(5,3)]	[(1,2), (3,2), (5,3)]
7	(5,3)	[(1,2), (3,2), (5,3)]	[(4,3), (5,4)]	[(1,2), (3,2), (4,3), (5,4)]
8	(5,4)	[(1,2), (3,2), (4,3), (5,4)]	[(5,5)]	[(1,2), (3,2), (4,3), (5,5)]
9	(5,5)	[(1,2), (3,2), (4,3), (5,5)]	[]	[(1,2), (3,2), (4,3)]

e.1 DFS Final Path

With the specified deterministic neighbor order, DFS returns:

$(1,1) \rightarrow (2,1) \rightarrow (3,1) \rightarrow (4,1) \rightarrow (5,1) \rightarrow (5,2) \rightarrow (5,3) \rightarrow (5,4) \rightarrow (5,5)$

Number of moves / total path cost = 8.

The DFS path is valid, but DFS is not an optimality-guaranteed algorithm. A different action/neighbor ordering could produce a different valid route and a different cost.

(f) Compare BFS and DFS for this specific scenario in terms of completeness, optimality, time complexity, and space complexity, and justify which algorithm is more suitable for this warehouse delivery problem.

Criterion	BFS	DFS	Specific conclusion for this problem
Completeness	Complete for a finite graph when implemented with a visited/discovered set.	Complete on this finite state space with visited-state checking; without visited checking, cycles can cause problems.	Both can find a solution here, but BFS gives a more systematic level-by-level search.
Optimality	Optimal for unit step costs.	Not guaranteed to be optimal.	BFS is optimal because each move costs 1; its path has cost 8.
Time complexity	$O(b^d)$ in the standard tree-search bound.	$O(b^m)$ in the standard tree-search bound.	Here the grid is tiny, so actual execution is fast; BFS's theoretical advantage is shortest-path discovery.
Space complexity	$O(b^d)$ because the frontier can contain a large level.	$O(bm)$ in the standard graph/tree-search bound for DFS stack behavior.	DFS generally uses less frontier memory; however, the 5×5 problem is small enough that BFS memory is negligible.
Main strength	Finds the shortest path in an unweighted/unit-cost graph.	Can reach deep solutions using relatively little frontier memory.	Shortest safe delivery is the key requirement, so BFS is preferable.
Main weakness	Can consume more memory because it stores a frontier level.	May follow a long route before discovering a better one.	DFS could return a non-minimal delivery route.

f.1 Standard Complexity Notation

Let b be the branching factor, d the depth of the shallowest goal, and m the maximum search depth. The standard worst-case tree-search bounds are:

- BFS time: $O(b^d)$; BFS space: $O(b^d)$.
- DFS time: $O(b^m)$; DFS space: $O(bm)$.
- For a finite graph with an explicit visited set, the actual number of unique states is bounded by the finite state space. Here there are only 20 non-blocked cells, so both searches are computationally small.

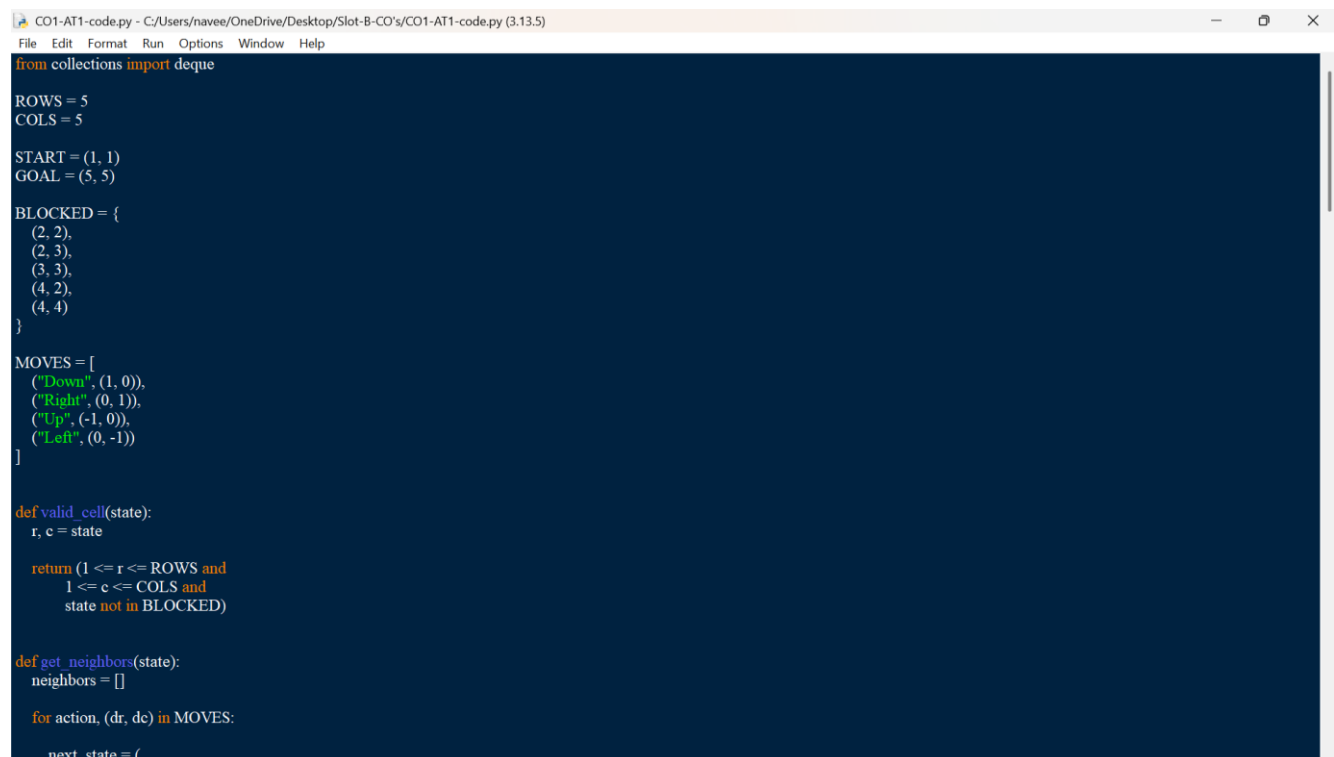
f.2 Best Algorithm

BFS is more suitable for the stated warehouse delivery problem. The movement cost is uniform, and the practical objective is to deliver the package using the shortest safe route. BFS explores all states at distance 0, then distance 1, then distance 2, and so on. Therefore, when it first reaches (5,5), no other undiscovered route can have fewer movements.

DFS is still useful as a general search method and can use less frontier memory, but its first solution is dependent on the chosen exploration order and is not guaranteed to minimize movement cost. For an autonomous delivery robot, unnecessary movements can increase travel time, energy consumption, and operational cost. Therefore, BFS is the safer choice when shortest unit-cost delivery is the stated objective.

Python Program – BFS and DFS for the Warehouse Robot

The following Python 3 program is directly executable in IDLE, VS Code, PyCharm, or a standard Python interpreter. It implements both BFS and DFS, avoids blocked cells, prevents revisiting already discovered states, prints the search order, and reports the final path and cost.



```
CO1-AT1-code.py - C:/Users/navee/OneDrive/Desktop/Slot-B-CO's/CO1-AT1-code.py (3.13.5)
File Edit Format Run Options Window Help
from collections import deque

ROWS = 5
COLS = 5

START = (1, 1)
GOAL = (5, 5)

BLOCKED = {
    (2, 2),
    (2, 3),
    (3, 3),
    (4, 2),
    (4, 4)
}

MOVES = [
    ("Down", (1, 0)),
    ("Right", (0, 1)),
    ("Up", (-1, 0)),
    ("Left", (0, -1))
]

def valid_cell(state):
    r, c = state
    return (1 <= r <= ROWS and
            1 <= c <= COLS and
            state not in BLOCKED)

def get_neighbors(state):
    neighbors = []
    for action, (dr, dc) in MOVES:
        next_state = (
```

```

    next_state = (state[0] + dr, state[1] + dc)

    if valid_cell(next_state):
        neighbors.append((next_state, action))

    return neighbors

def reconstruct_path(parent, state):
    path = []

    while state is not None:
        path.append(state)
        state = parent[state]

    path.reverse()
    return path

def bfs():
    queue = deque([START])

    parent = {START: None}

    expansion = 0

    print("\n===== BFS =====")

    while queue:
        print("Queue before expansion:", list(queue))

        current = queue.popleft()
        expansion += 1

        print("Expanded:", current)

        if current == GOAL:

```

```

            path = reconstruct_path(parent, current)
            print("Goal reached!")
            print("BFS Path:", path)
            print("BFS Cost:", len(path) - 1)
            print("BFS Expansions:", expansion)
            return path

    for next_state, action in get_neighbors(current):
        if next_state not in parent:
            parent[next_state] = current
            queue.append(next_state)
            print(" Added:", next_state, "using", action)

    print("Queue after expansion:", list(queue))
    print()

    return None

def dfs():
    stack = [START]

    parent = {START: None}

    expansion = 0

    print("\n===== DFS =====")

    while stack:
        print("Stack before expansion:", stack)

        current = stack.pop()
        expansion += 1

        print("Expanded:", current)

        if current == GOAL:

```

```

    path = reconstruct_path(parent, current)
    print("Goal reached!")
    print("DFS Path:", path)
    print("DFS Cost:", len(path) - 1)
    print("DFS Expansions:", expansion)
    return path

neighbors = get_neighbors(current)

for next_state, action in reversed(neighbors):
    if next_state not in parent:
        parent[next_state] = current
        stack.append(next_state)
        print(" Pushed:", next_state, "using", action)

    print("Stack after expansion:", stack)
    print()

return None

# Run both searches
bfs_path = bfs()
dfs_path = dfs()

print("\n===== FINAL COMPARISON =====")

if bfs_path:
    print("BFS total cost:", len(bfs_path) - 1)
else:
    print("BFS found no path.")

if dfs_path:
    print("DFS total cost:", len(dfs_path) - 1)
else:
    print("DFS found no path.")

```

Ln: 151 Col: 31

Output:

```

IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/navee/OneDrive/Desktop/Slot-B-CO's/CO1-AT1-code.py =====

BREADTH-FIRST SEARCH

Queue before expansion:
deque([(1, 1)])
Expanded: (1, 1)
Added: (2, 1) using Down
Added: (1, 2) using Right
Queue after expansion:
deque([(2, 1), (1, 2)])

Queue before expansion:
deque([(2, 1), (1, 2)])
Expanded: (2, 1)
Added: (3, 1) using Down
Queue after expansion:
deque([(1, 2), (3, 1)])

Queue before expansion:
deque([(1, 2), (3, 1)])
Expanded: (1, 2)
Added: (1, 3) using Right
Queue after expansion:
deque([(3, 1), (1, 3)])

Queue before expansion:
deque([(3, 1), (1, 3)])
Expanded: (3, 1)
Added: (4, 1) using Down
Added: (3, 2) using Right
Queue after expansion:
deque([(1, 3), (4, 1), (3, 2)])

Queue before expansion:

```

```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help

deque([(1, 3), (4, 1), (3, 2)])
Expanded: (1, 3)
Added: (1, 4) using Right
Queue after expansion:
deque([(4, 1), (3, 2), (1, 4)])

Queue before expansion:
deque([(4, 1), (3, 2), (1, 4)])
Expanded: (4, 1)
Added: (5, 1) using Down
Queue after expansion:
deque([(3, 2), (1, 4), (5, 1)])

Queue before expansion:
deque([(3, 2), (1, 4), (5, 1)])
Expanded: (3, 2)
Queue after expansion:
deque([(1, 4), (5, 1)])

Queue before expansion:
deque([(1, 4), (5, 1)])
Expanded: (1, 4)
Added: (2, 4) using Down
Added: (1, 5) using Right
Queue after expansion:
deque([(5, 1), (2, 4), (1, 5)])

Queue before expansion:
deque([(5, 1), (2, 4), (1, 5)])
Expanded: (5, 1)
Added: (5, 2) using Right
Queue after expansion:
deque([(2, 4), (1, 5), (5, 2)])

Queue before expansion:
deque([(2, 4), (1, 5), (5, 2)])
Expanded: (2, 4)
Added: (3, 4) using Down
```

```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help

Added: (3, 4) using Down
Added: (2, 5) using Right
Queue after expansion:
deque([(1, 5), (5, 2), (3, 4), (2, 5)])

Queue before expansion:
deque([(1, 5), (5, 2), (3, 4), (2, 5)])
Expanded: (1, 5)
Queue after expansion:
deque([(5, 2), (3, 4), (2, 5)])

Queue before expansion:
deque([(5, 2), (3, 4), (2, 5)])
Expanded: (5, 2)
Added: (5, 3) using Right
Queue after expansion:
deque([(3, 4), (2, 5), (5, 3)])

Queue before expansion:
deque([(3, 4), (2, 5), (5, 3)])
Expanded: (3, 4)
Added: (3, 5) using Right
Queue after expansion:
deque([(2, 5), (5, 3), (3, 5)])

Queue before expansion:
deque([(2, 5), (5, 3), (3, 5)])
Expanded: (2, 5)
Queue after expansion:
deque([(5, 3), (3, 5)])

Queue before expansion:
deque([(5, 3), (3, 5)])
Expanded: (5, 3)
Added: (5, 4) using Right
Added: (4, 3) using Up
Queue after expansion:
deque([(3, 5), (5, 4), (4, 3)])
```

```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help

Queue after expansion:
deque([(3, 5), (5, 4), (4, 3)])

Queue before expansion:
deque([(3, 5), (5, 4), (4, 3)])
Expanded: (3, 5)
Added: (4, 5) using Down
Queue after expansion:
deque([(5, 4), (4, 3), (4, 5)])

Queue before expansion:
deque([(5, 4), (4, 3), (4, 5)])
Expanded: (5, 4)
Added: (5, 5) using Right
Queue after expansion:
deque([(4, 3), (4, 5), (5, 5)])

Queue before expansion:
deque([(4, 3), (4, 5), (5, 5)])
Expanded: (4, 3)
Queue after expansion:
deque([(4, 5), (5, 5)])

Queue before expansion:
deque([(4, 5), (5, 5)])
Expanded: (4, 5)
Queue after expansion:
deque([(5, 5)])

Queue before expansion:
deque([(5, 5)])
Expanded: (5, 5)

Goal reached!
BFS Path:
[(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)]
BFS Cost:
8
```

```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help

DEPTH-FIRST SEARCH

Stack before expansion:
[(1, 1)]
Expanded: (1, 1)
Pushed: (1, 2) using Right
Pushed: (2, 1) using Down
Stack after expansion:
[(1, 2), (2, 1)]

Stack before expansion:
[(1, 2), (2, 1)]
Expanded: (2, 1)
Pushed: (3, 1) using Down
Stack after expansion:
[(1, 2), (3, 1)]

Stack before expansion:
[(1, 2), (3, 1)]
Expanded: (3, 1)
Pushed: (3, 2) using Right
Pushed: (4, 1) using Down
Stack after expansion:
[(1, 2), (3, 2), (4, 1)]

Stack before expansion:
[(1, 2), (3, 2), (4, 1)]
Expanded: (4, 1)
Pushed: (5, 1) using Down
Stack after expansion:
[(1, 2), (3, 2), (5, 1)]

Stack before expansion:
[(1, 2), (3, 2), (5, 1)]
Expanded: (5, 1)
Pushed: (5, 2) using Right
Stack after expansion:
[(1, 2), (3, 2), (5, 2)]
```

```

IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help

[(1, 2), (3, 2), (5, 3)]
Expanded: (5, 3)
Pushed: (4, 3) using Up
Pushed: (5, 4) using Right
Stack after expansion:
[(1, 2), (3, 2), (4, 3), (5, 4)]

Stack before expansion:
[(1, 2), (3, 2), (4, 3), (5, 4)]
Expanded: (5, 4)
Pushed: (5, 5) using Right
Stack after expansion:
[(1, 2), (3, 2), (4, 3), (5, 5)]

Stack before expansion:
[(1, 2), (3, 2), (4, 3), (5, 5)]
Expanded: (5, 5)

Goal reached!
DFS Path:
[(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)]
DFS Cost:
8
DFS Nodes Expanded:
9

FINAL COMPARISON

BFS Path:
[(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)]
BFS Total Cost:
8

DFS Path:
[(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)]
DFS Total Cost:
8

```

Results

Algorithm	Path	Total Cost	Optimality
BFS	(1,1) → (2,1) → (3,1) → (4,1) → (5,1) → (5,2) → (5,3) → (5,4) → (5,5)	8	Optimal for unit-cost moves
DFS	(1,1) → (2,1) → (3,1) → (4,1) → (5,1) → (5,2) → (5,3) → (5,4) → (5,5)	8	Not guaranteed optimal

Note: The exact DFS route depends on the neighbor/action ordering used by the implementation. The DFS result reported in this document corresponds specifically to the deterministic order Down, Right, Up, Left, with neighbors pushed in reverse order because the stack is LIFO.

Conclusion

The warehouse delivery robot is naturally modeled as a goal-directed intelligent agent operating in a discrete, deterministic, static, and fully observable grid environment. Its PEAS description captures the delivery objective, warehouse environment, movement/delivery actuators, and position/obstacle sensors. Among the agent categories provided, a Goal-Based Agent is most appropriate because the robot must deliberately plan actions to reach a specified destination.

The formal search problem has (1,1) as the initial state and (5,5) as the goal, with four possible movement actions restricted by grid boundaries and blocked cells. BFS systematically explores the state space by increasing depth and, because every action costs 1, returns a shortest path. DFS can also find a route in this finite problem, but its solution quality depends on the search order and it does not guarantee minimum cost. Consequently, BFS is the more suitable search algorithm when shortest safe delivery is the primary requirement.